

Linear List Search

1 Problem

Linear List Search

We have a list of n items. A request asks for one item. If the item is in position k , then accessing it costs k .

The input is a request sequence

$$\sigma = (x_1, x_2, \dots, x_m).$$

After serving a request, the algorithm may reorder the list. The goal is to minimize the total access cost.

Online Difficulty

The algorithm sees only the current request. It does not know which items will be requested later.

2 Example

Suppose the list is

$$(1, 2, 3, 4, 5).$$

If the request is 4, then the access cost is

$$4.$$

After accessing 4, an online algorithm may move it closer to the front, hoping it will be requested again.

3 Strategy 1: Move Forward by 2

Move Forward by 2

After accessing an item, move it two positions closer to the front of the list.

This strategy looks reasonable, but it can perform very badly.

Bad Request Sequence

Initial list:

$$(1, 2, 3, \dots, n).$$

Request the last item repeatedly:

$$\sigma = (n, n, n, \dots, n).$$

The first access costs n . After that, the item moves forward by only two positions. So the first few costs are

$$n + (n - 2) + (n - 4) + \dots$$

For about $n/2$ requests, the accessed item is still moving toward the front. The total cost is therefore quadratic:

$$n + (n - 2) + (n - 4) + \dots = cn^2$$

for some positive constant c . The exact constant is not important. What matters is that the cost grows like n^2 .

The offline optimum knows that n will be requested repeatedly. It can pay n once, move n to the front, and then pay 1 for later repeated requests:

$$n + 1 + 1 + \dots + 1 = 2n - 1 < 2n.$$

Competitive Ratio

The ratio is at least

$$\frac{\text{online cost}}{\text{offline cost}} > \frac{cn^2}{2n} = \frac{c}{2}n.$$

As

$$n \rightarrow \infty,$$

we get

$$\frac{c}{2}n \rightarrow \infty.$$

Conclusion

Move-forward-by-2 is not competitive. Its competitive ratio becomes unbounded as n becomes large.

4 Strategy 2: Move To Front

Move To Front

The **Move-To-Front** strategy, abbreviated MTF, moves the requested item to the front immediately after accessing it.

For example, if the list is

$$(1, 2, 3, 4, 5)$$

and the request is 4, then the cost is 4, and the new list becomes

$$(4, 1, 2, 3, 5).$$

Main Theorem

MTF is 2-competitive.

5 Why MTF Works

MTF is good because it reacts strongly to repeated requests. If an item is requested once, it moves to the front, so another request for the same item becomes cheap.

The proof compares two algorithms:

Algorithm	Meaning
MTF list	The list maintained by the online algorithm.
OPT list	The list maintained by the optimal offline algorithm.

At the beginning, assume both lists are the same:

$$\text{OPT} : (1, 2, 3, \dots, n), \quad \text{MTF} : (1, 2, 3, \dots, n).$$

At request j , suppose the requested item is i . If i is at position k in the MTF list, then MTF pays cost k .

6 Inversions Between Two Lists

Inversion

An **inversion** is a pair of items whose relative order is different in the MTF list and in the OPT list.

Example:

$$\begin{aligned} \text{OPT} : & \quad \dots a \dots b \dots \\ \text{MTF} : & \quad \dots b \dots a \dots \end{aligned}$$

The pair (a, b) is an inversion because the two lists disagree about which item comes first.

Let

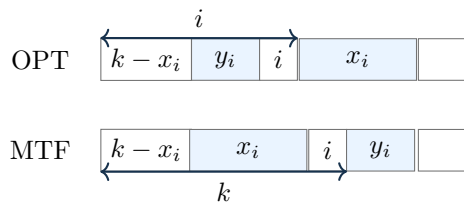
$$\text{INV}(j)$$

be the number of inversions before request j .

7 The x_i and y_i Groups

For the requested item i , define:

Symbol	Meaning
x_i	Number of items after i in OPT but before i in MTF.
y_i	Number of items before i in OPT but after i in MTF.



The x_i items are before i in MTF, so MTF pays for passing them. In OPT, those same items are after i .

8 Amortized Cost

Suppose MTF accesses item x at position k . Its real cost is k .

Instead of analyzing only the real cost, define the amortized cost:

$$T(j) = k + \text{INV}(j+1) - \text{INV}(j).$$

This means:

- pay the real access cost k ;
- add the change in inversion count.

Why Use Inversions?

The inversion count measures how different MTF's list is from OPT's list. If MTF moves an item to the front and becomes closer to OPT, the inversion count decreases and helps pay for the move.

9 One-Step MTF Bound

When MTF moves item i to the front, the inversion count changes.

- x_i old inversions disappear.
- At most $k - x_i$ new inversions appear.
- Let $\alpha \geq 0$ be the number of additional inversions removed by OPT's own reordering.

Thus the amortized cost satisfies

$$T(j) = k + ((k - x_i) + y_i - \alpha) - (x_i + y_i).$$

Simplify:

$$T(j) = 2k - 2x_i - \alpha.$$

Since $\alpha \geq 0$,

$$T(j) \leq 2k - 2x_i.$$

So

$$T(j) \leq 2(k - x_i).$$

From the diagram, OPT's position of item i is at least $k - x_i$. Therefore

$$k - x_i \leq \text{OPT cost for request } j.$$

Hence

$$T(j) \leq 2 \cdot \text{OPT cost for request } j.$$

10 Telescoping the Amortized Costs

For each request:

$$\begin{aligned} T(1) &= k_1 + \text{INV}(2) - \text{INV}(1), \\ T(2) &= k_2 + \text{INV}(3) - \text{INV}(2), \\ &\dots \\ T(m) &= k_m + \text{INV}(m+1) - \text{INV}(m). \end{aligned}$$

Now sum all requests:

$$\sum_{j=1}^m T(j) \leq 2 \text{OPT}(\sigma).$$

But the amortized costs telescope:

$$\sum_{j=1}^m T(j) = \sum_{j=1}^m k_j + \text{INV}(m+1) - \text{INV}(1).$$

Here k_j is the real cost paid by MTF on request j . Therefore the total real cost of MTF is

$$C_{\text{MTF}}(\sigma) = \sum_{j=1}^m k_j.$$

The initial lists are the same, so there are no inversions at the beginning:

$$\text{INV}(1) = 0.$$

Also, an inversion count can never be negative:

$$\text{INV}(m+1) \geq 0.$$

So the telescoping equation becomes

$$\sum_{j=1}^m T(j) = \sum_{j=1}^m k_j + \text{INV}(m+1).$$

The right side is the real MTF cost plus a nonnegative extra term. Therefore

$$\sum_{j=1}^m k_j \leq \sum_{j=1}^m T(j).$$

Earlier we proved each request satisfies

$$T(j) \leq 2 \cdot \text{OPT cost on request } j.$$

Adding this inequality over all requests gives

$$\sum_{j=1}^m T(j) \leq 2 \text{OPT}(\sigma).$$

Combining the two bounds:

$$C_{\text{MTF}}(\sigma) = \sum_{j=1}^m k_j \leq \sum_{j=1}^m T(j) \leq 2 \text{OPT}(\sigma).$$

Hence

$$C_{\text{MTF}}(\sigma) \leq 2 \text{OPT}(\sigma).$$

Result

$$C_{\text{MTF}}(\sigma) \leq 2 \text{OPT}(\sigma).$$

Therefore, MTF is 2-competitive.

11 Why No Online Algorithm Can Do Much Better

There is a lower bound for linear list search:

Lower Bound

No deterministic online algorithm for linear list search has competitive ratio better than

$$2 - \frac{2}{n+1}.$$

This means MTF's ratio 2 is almost optimal when n is large.

12 Lower Bound Idea

Consider any online algorithm A . An adversary can choose requests maliciously.

At each step, the adversary requests the item currently at the last position in A 's list. Therefore, A pays

$$n$$

on every request. For m requests,

$$C_A(\sigma) = mn.$$

Now compare this with an offline algorithm. The offline algorithm knows the whole request sequence

$$\sigma = x_1, x_2, \dots, x_m$$

in advance. We do not need to know the exact optimal offline strategy. It is enough to show that there exists one fixed list order whose cost is small, because OPT can only be better than that fixed order.

Best Fixed Permutation Argument

Generate all possible fixed permutations of the n items:

$$\pi_1, \pi_2, \dots, \pi_{n!}.$$

For the given request sequence σ , let $O(\pi_i)$ be the total search cost if we use permutation π_i and never change it. The best fixed permutation has cost

$$\min_i O(\pi_i).$$

Since a minimum is always at most the average,

$$\min_i O(\pi_i) \leq \frac{O(\pi_1) + O(\pi_2) + \dots + O(\pi_{n!})}{n!}.$$

Now compute that average. Look at one request x_j . Across all $n!$ permutations, the requested item appears:

$(n-1)!$ times in position 1,

$(n-1)!$ times in position 2,

and so on up to position n . Therefore the total cost of searching this one requested item over all permutations is

$$(n-1)!(1 + 2 + \cdots + n).$$

Using

$$1 + 2 + \cdots + n = \frac{n(n+1)}{2},$$

we get

$$(n-1)! \frac{n(n+1)}{2} = \frac{n!(n+1)}{2}.$$

So the average cost of this one request over all $n!$ permutations is

$$\frac{n!(n+1)/2}{n!} = \frac{n+1}{2}.$$

There are m requests, so the average total cost of a fixed permutation is

$$\frac{m(n+1)}{2}.$$

Hence at least one fixed permutation has cost at most

$$\frac{m(n+1)}{2}.$$

Since OPT is the best offline algorithm,

$$\text{OPT}(\sigma) \leq \min_i O(\pi_i) \leq \frac{m(n+1)}{2}.$$

Thus

$$\frac{C_A(\sigma)}{\text{OPT}(\sigma)} \geq \frac{mn}{m(n+1)/2} = \frac{2n}{n+1} = 2 - \frac{2}{n+1}.$$

Meaning

For large n ,

$$2 - \frac{2}{n+1} \approx 2.$$

So no online algorithm can guarantee a ratio significantly below 2, and MTF is essentially best possible.

13 Strategy Comparison

Strategy	Competitive behavior	Reason
Move forward by 2	Not competitive	Repeated requests to a deep item remain expensive for many steps.
Move to front	2-competitive	Repeatedly requested items become cheap immediately.
Best possible online ratio	At least $2 - \frac{2}{n+1}$	Adversary can always request the current last item.

14 Review Box

Core Facts

Access cost = position of requested item in the list.

Move-forward-by-2 is not competitive.

MTF moves each requested item to the front.

MTF is 2-competitive.

No deterministic online algorithm can beat $2 - \frac{2}{n+1}$.

For large n , this lower bound is close to 2.